1.)

Step	Operation	Explanation	Resulting array
1	Check node 1 (value= 27)	Left child = 17, right child = 3.27 is largest → No swaps	{27, 17, 3, 16, 13, 10, 1, 5, 7, 12, 4, 8, 9, 0}
Final result		No changes occur because the root satisfies the max heap property	{27, 17, 3, 16, 13, 10, 1, 5, 7, 12, 4, 8, 9, 0}

2.)

step	operation	Action/explanation	Resulting array
1	build-Max-Heap(A)	transform array into max heap	{25, 13, 20, 8, 7, 17, 2, 5, 4}
2	Swap A[1] ←→ A[9]	Move max (25) to end	{4, 13, 20, 8, 7, 17, 2, 5, 25}
3	MAX-HEAPIFY(A, 1)	restore heap	{20, 13, 17, 8, 7, 4, 2, 5, 25}
4	swap A[1] ←→ A[8]	move max (20) to end	{5, 13, 17, 8, 7, 4, 2, 20, 25}
5	MAX-HEAPIFY(A, 1)	restore heap	{17, 13, 5, 8, 7, 4, 2, 20, 25}
6	swap A[1] ←→ A[7]	move max (17) to end	{2, 13, 5, 8, 7, 4, 17, 20, 25}
7	MAX-HEAPIFY(A, 1)	restore heap	{13, 8, 5, 2, 7, 4, 17, 20, 25}
8	Swap A[1] ←→ A[6]	move max (13) to end	{4, 8, 5, 2, 7, 13, 17, 20, 25}
9	MAX-HEAPIFY(A, 1)	restore heap	{8, 7, 5, 2, 4, 13, 17, 20, 25}
10	swap A[1] ←→ A[5]	move max (8) to end	{4, 7, 5, 2, 8, 13, 17, 20, 25}
11	MAX-HEAPIFY(A, 1)	Restore heap	{7, 4, 5, 2, 8, 13, 17, 20, 25}
12	Swap A[1] ←→ A[4]	move max (7) to end	{2, 4, 5, 7, 8, 13, 17, 20, 25}
13	MAX-HEAPIFY(A, 1)	restore heap	{5, 4, 2, 7, 8, 13, 17, 20, 25}
14	continue until size = 1	final sorted array	{2, 4, 5, 7, 8, 13, 17, 20, 25}

3.)

a) about 16 swaps. First it builds a max heap then repeatedly swaps the root with the last element in the heap and calls the MAX-HEAPIFY(A,1) to restore the heap. After each loop it swaps between A[1] and A[i], and the swaps inside both call to MAX-HEAPIFY. So it takes around 16 swaps, one per outer iteration and multiple from the internal heapify adjustments.

	Heap repair step	Swap count	efficiency	Explanation
a) Normal heapsort	MAX-HEAPIFY(A,1)	About 16	O(n log n)	Fixes one brach of heap only

b) a lot bigger. If we use Build-MAX-Heap(A) instead of MAX-HEAPIFY(A, 1), the program will rebuild the whole heap every time instead of just fixing the top part. This means it does a lot more swaps because it goes through almost all elements again and again. So the total swap count would be around O(n^2) overall. It would still sort the array correctly but it would take much longer since it repeats extra work each loop.

	Heap repair step	Swap count	efficiency	Explanation
b) Modified heapsort	Build-MAX-Heap(A)	A lot bigger	O(n^2)	Rebuilds entire heap each time